
Ping and Traceroute

Release 1.0

Abdul Aleem Shaik Gadda

Oct 09, 2026

CONTENTS

1	My Ping	1
2	My Traceroute	7
3	Utilities	14
	Python Module Index	17
	Index	18

MY PING

A simple implementation of the ping command (macOS-style output).

Sends ICMP Echo Requests over a raw socket and prints the replies. Requires root/administrator privileges.

Example:

```
sudo python3 my_ping.py -c 4 google.com
```

`my_ping.parse_args()`

Parse command-line arguments.

Returns

Parsed arguments.

Return type

`argparse.Namespace`

Source code:

```
def parse_args():
    """Parse command-line arguments.

    :return: Parsed arguments.
    :rtype: argparse.Namespace
    """
    parser = argparse.ArgumentParser(
        description="Send ICMP ECHO_REQUEST packets to a host.")
    parser.add_argument("host", help="destination hostname or IP address")
    parser.add_argument("-c", dest="count", type=int, default=None,
                        help="stop after sending COUNT packets")
    parser.add_argument("-i", dest="wait", type=float, default=1.0,
                        help="seconds to wait between packets (default 1)
↪")
    parser.add_argument("-s", dest="size", type=int, default=56,
                        help="number of data bytes to send (default 56)")
    parser.add_argument("-t", dest="timeout", type=float, default=None,
                        help="exit after this many seconds")
    args = parser.parse_args()
    if args.count is not None and args.count < 1:
        parser.error("bad number of packets to transmit")
```

(continues on next page)

(continued from previous page)

```

if args.wait <= 0:
    parser.error("bad wait time")
if not 0 <= args.size <= 65507 - ICMP_HEADER_LEN:
    parser.error("illegal packet size")
return args

```

`my_ping.make_payload(size)`

Create a payload of size bytes with an incrementing pattern.

Parameters

size (*int*) – Number of data bytes.

Returns

Payload bytes.

Return type

bytes

Source code:

```

def make_payload(size):
    """Create a payload of ``size`` bytes with an incrementing pattern.

    :param int size: Number of data bytes.
    :return: Payload bytes.
    :rtype: bytes
    """
    return bytes((i + 8) % 256 for i in range(size))

```

`my_ping.wait_for_reply(sock, ident, seq, deadline)`

Wait for the Echo Reply that matches ident and seq.

Parameters

- **sock** (*socket.socket*) – Raw ICMP socket.
- **ident** (*int*) – Expected ICMP identifier.
- **seq** (*int*) – Expected sequence number.
- **deadline** (*float*) – `time.perf_counter()` value to give up at.

Returns

(*arrival_time*, *source_ip*, *ttl*, *icmp_length*) or `None`.

Return type

tuple or `None`

Source code:

```

def wait_for_reply(sock, ident, seq, deadline):
    """Wait for the Echo Reply that matches ``ident`` and ``seq``.

    :param socket.socket sock: Raw ICMP socket.
    :param int ident: Expected ICMP identifier.

```

(continues on next page)

(continued from previous page)

```

:param int seq: Expected sequence number.
:param float deadline: ``time.perf_counter()`` value to give up at.
:return: ``(arrival_time, source_ip, ttl, icmp_length)`` or ``None``.
:rtype: tuple or None
"""
while True:
    remaining = deadline - time.perf_counter()
    if remaining <= 0:
        return None
    ready, _, _ = select.select([sock], [], [], remaining)
    if not ready:
        return None
    data, _ = sock.recvfrom(65535)
    arrival = time.perf_counter()
    if len(data) < IP_HEADER_LEN + ICMP_HEADER_LEN:
        continue
    header_len, ttl, src = parse_ip_header(data)
    icmp = data[header_len:]
    if len(icmp) < ICMP_HEADER_LEN:
        continue
    r_type, _, _, r_id, r_seq = struct.unpack("!BBHHH", icmp[:8])
    if r_type == ICMP_ECHO_REPLY and r_id == ident and r_seq == seq:
        return arrival, src, ttl, len(icmp)

```

`my_ping.print_stats(host, sent, rtts)`

Print the summary shown when ping finishes.

Parameters

- **host** (*str*) – Destination as typed by the user.
- **sent** (*int*) – Packets transmitted.
- **rtts** (*list*) – Round-trip times in milliseconds.

Source code:

```

def print_stats(host, sent, rtts):
    """Print the summary shown when ping finishes.

    :param str host: Destination as typed by the user.
    :param int sent: Packets transmitted.
    :param list rtts: Round-trip times in milliseconds.
    """
    received = len(rtts)
    loss = 100.0 * (sent - received) / sent if sent else 0.0
    print(f"\n--- {host} ping statistics ---")
    print(f"{sent} packets transmitted, {received} packets received, "
          f"{loss:.1f}% packet loss")
    if rtts:
        avg = sum(rtts) / received
        mdev = math.sqrt(max(sum(r * r for r in rtts) / received

```

(continues on next page)

(continued from previous page)

```

        - avg * avg, 0.0))
    print(f"round-trip min/avg/max/stddev = {min(rtts):.3f}/{avg:.3f}
↪ /"
        f"{max(rtts):.3f}/{mdev:.3f} ms")

```

my_ping.main()

Run the ping program.

Returns

Process exit status (0 if any reply was received).

Return type

int

Source code:

```

def main():
    """Run the ping program.

    :return: Process exit status (0 if any reply was received).
    :rtype: int
    """
    args = parse_args()
    try:
        dest_ip = resolve_host(args.host)
    except socket.gaierror:
        print(f"ping: cannot resolve {args.host}: Unknown host",
              file=sys.stderr)
        return 68
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_RAW,
                             socket.IPPROTO_ICMP)
    except PermissionError:
        print("ping: raw sockets need root/administrator privileges "
              "(try sudo)", file=sys.stderr)
        return 1

    ident = os.getpid() & 0xFFFF
    payload = make_payload(args.size)
    print(f"PING {args.host} ({dest_ip}): {args.size} data bytes",
          flush=True)

    start = time.perf_counter()
    stop_at = start + args.timeout if args.timeout is not None else None
    rtts = []
    sent = 0
    seq = -1          # macOS-style: first icmp_seq is 0
    pending = None    # seq of the previous packet that got no reply yet
    try:
        while args.count is None or sent < args.count:
            if stop_at is not None and time.perf_counter() >= stop_at:

```

(continues on next page)

(continued from previous page)

```

        break
    seq += 1
    packet = build_echo_request(ident, seq & 0xFFFF, payload)
    send_time = time.perf_counter()
    send_ok = True
    try:
        sock.sendto(packet, (dest_ip, 0))
    except OSError as exc:
        print(f"ping: sendto: {exc.strerror}", flush=True)
        send_ok = False
    sent += 1

    # Report the previous packet as timed out once the next is
    ↪ sent.
    if pending is not None:
        print(f"Request timeout for icmp_seq {pending}",
    ↪ flush=True)
        pending = None

    interval_end = send_time + args.wait
    if send_ok:
        deadline = max(interval_end, send_time + 1.0)
        if stop_at is not None:
            deadline = min(deadline, stop_at)
        result = wait_for_reply(sock, ident, seq & 0xFFFF,
    ↪ deadline)
        if result:
            arrival, src, ttl, length = result
            rtt = (arrival - send_time) * 1000
            rtt.append(rtt)
            print(f"{length} bytes from {src}: icmp_seq={seq} "
                  f"ttl={ttl} time={rtt:.3f} ms", flush=True)
        else:
            pending = seq
    else:
        pending = seq

    # Sleep out the rest of the interval before the next packet.
    if args.count is None or sent < args.count:
        pause = interval_end - time.perf_counter()
        if stop_at is not None:
            pause = min(pause, stop_at - time.perf_counter())
        if pause > 0:
            time.sleep(pause)
    except KeyboardInterrupt:
        pass
    finally:
        sock.close()

```

(continues on next page)

(continued from previous page)

```
print_stats(args.host, sent, rtts)
return 0 if rtts else 1
```

MY TRACEROUTE

A simple implementation of the `traceroute` command (macOS-style output).

UDP probes are sent with an increasing TTL; routers along the path reply with ICMP Time Exceeded and the destination replies with ICMP Port Unreachable. Requires root/administrator privileges.

Example:

```
sudo python3 my_traceroute.py -n -q 2 google.com
```

`my_traceroute.parse_args()`

Parse command-line arguments.

Returns

Parsed arguments.

Return type

`argparse.Namespace`

Source code:

```
def parse_args():
    """Parse command-line arguments.

    :return: Parsed arguments.
    :rtype: argparse.Namespace
    """
    parser = argparse.ArgumentParser(
        description="Print the route packets take to a network host.")
    parser.add_argument("host", help="destination hostname or IP address")
    parser.add_argument("-n", dest="numeric", action="store_true",
                        help="print hop addresses numerically only")
    parser.add_argument("-q", dest="nqueries", type=int, default=3,
                        help="number of probes per TTL (default 3)")
    parser.add_argument("-S", dest="summary", action="store_true",
                        help="print a summary of unanswered probes per hop")
    args = parser.parse_args()
    if not 1 <= args.nqueries <= 10:
        parser.error("number of queries must be between 1 and 10")
    return args
```

`my_traceroute.probe_port_from_icmp(icmp)`

Extract the original UDP destination port from an ICMP error.

The error payload holds the original IP header plus the first 8 bytes of the original datagram (the UDP header).

Parameters

icmp (*bytes*) – ICMP message (IP header already removed).

Returns

The destination port of the probe, or `None`.

Return type

`int` or `None`

Source code:

```
def probe_port_from_icmp(icmp):
    """Extract the original UDP destination port from an ICMP error.

    The error payload holds the original IP header plus the first 8 bytes
    of the original datagram (the UDP header).

    :param bytes icmp: ICMP message (IP header already removed).
    :return: The destination port of the probe, or ``None``.
    :rtype: int or None
    """
    if len(icmp) < 8 + 20:
        return None
    inner = icmp[8:]
    inner_len = (inner[0] & 0x0F) * 4
    udp = inner[inner_len:inner_len + 8]
    if len(udp) < 8:
        return None
    return struct.unpack("!HH", udp[:4])[1]
```

`my_traceroute.wait_for_icmp(recv_sock, port, deadline)`

Wait for the ICMP error caused by the probe sent to `port`.

Parameters

- **recv_sock** (*socket.socket*) – Raw ICMP socket.
- **port** (*int*) – UDP destination port of the probe.
- **deadline** (*float*) – `time.perf_counter()` value to give up at.

Returns

(`arrival_time`, `source_ip`, `icmp_type`, `icmp_code`) or `None`.

Return type

`tuple` or `None`

Source code:

```
def wait_for_icmp(recv_sock, port, deadline):
    """Wait for the ICMP error caused by the probe sent to ``port``.
```

(continues on next page)

(continued from previous page)

```

:param socket.socket recv_sock: Raw ICMP socket.
:param int port: UDP destination port of the probe.
:param float deadline: ``time.perf_counter()`` value to give up at.
:return: ``(arrival_time, source_ip, icmp_type, icmp_code)`` or None.
:rtype: tuple or None
"""
while True:
    remaining = deadline - time.perf_counter()
    if remaining <= 0:
        return None
    ready, _, _ = select.select([recv_sock], [], [], remaining)
    if not ready:
        return None
    data, _ = recv_sock.recvfrom(65535)
    arrival = time.perf_counter()
    if len(data) < 20 + 8:
        continue
    header_len, _, src = parse_ip_header(data)
    icmp = data[header_len:]
    icmp_type, icmp_code = icmp[0], icmp[1]
    if icmp_type not in (ICMP_TIME_EXCEEDED, ICMP_DEST_UNREACHABLE):
        continue
    if probe_port_from_icmp(icmp) == port:
        return arrival, src, icmp_type, icmp_code

```

`my_traceroute.describe(addr, numeric, cache)`

Format a router address for display.

Parameters

- **addr** (*str*) – IPv4 address.
- **numeric** (*bool*) – If true, show only the IP address.
- **cache** (*dict*) – Reverse-DNS cache.

Returns

"name (ip)" or just "ip" with -n.

Return type

str

Source code:

```

def describe(addr, numeric, cache):
    """Format a router address for display.

    :param str addr: IPv4 address.
    :param bool numeric: If true, show only the IP address.
    :param dict cache: Reverse-DNS cache.
    :return: ``"name (ip)``" or just ``"ip"`` with ``-n``.
    :rtype: str
    """

```

(continues on next page)

(continued from previous page)

```

"""
if numeric:
    return addr
if addr not in cache:
    try:
        cache[addr] = socket.gethostbyaddr(addr)[0]
    except (socket.herror, socket.gaierror, OSError):
        cache[addr] = addr
return f"{cache[addr]} ({addr})"

```

`my_traceroute.trace_hop(ttl, dest_ip, args, send_sock, recv_sock, state)`

Send all probes for one TTL and print the hop line.

Parameters

- **ttl** (*int*) – Current time-to-live value.
- **dest_ip** (*str*) – Destination IPv4 address.
- **args** (*argparse.Namespace*) – Parsed arguments.
- **send_sock** (*socket.socket*) – UDP socket used to send probes.
- **recv_sock** (*socket.socket*) – Raw ICMP socket.
- **state** (*dict*) – Mutable state holding port and cache.

Returns

(reached_destination, unanswered_probes).

Return type

tuple

Source code:

```

def trace_hop(ttl, dest_ip, args, send_sock, recv_sock, state):
    """Send all probes for one TTL and print the hop line.

    :param int ttl: Current time-to-live value.
    :param str dest_ip: Destination IPv4 address.
    :param argparse.Namespace args: Parsed arguments.
    :param socket.socket send_sock: UDP socket used to send probes.
    :param socket.socket recv_sock: Raw ICMP socket.
    :param dict state: Mutable state holding ``port`` and ``cache``.
    :return: ``(reached_destination, unanswered_probes)``.
    :rtype: tuple
    """
    send_sock.setsockopt(socket.IPPROTO_IP, socket.IP_TTL, ttl)
    last_addr = None
    done = False
    lost = 0
    for probe in range(args.nqueries):
        port = state["port"]
        state["port"] += 1

```

(continues on next page)

(continued from previous page)

```

start = time.perf_counter()
send_failed = False
try:
    send_sock.sendto(PAYLOAD, (dest_ip, port))
except OSError as exc:
    print(f"traceroute: sendto: {exc.strerror}", file=sys.stderr,
          flush=True)
    send_failed = True

# The hop number is printed after the first probe is sent, which
↪ is
# why the sendto error for probe 1 appears on the line above it.
if probe == 0:
    print(f"{ttl:2d} ", end="", flush=True)
if send_failed:
    print(f"traceroute: wrote {args.host} {PACKET_LEN} chars, "
          f"ret=-1", file=sys.stderr, flush=True)

result = wait_for_icmp(recv_sock, port, start + PROBE_TIMEOUT)
if result is None:
    lost += 1
    print(" *", end="", flush=True)
    continue
arrival, src, icmp_type, icmp_code = result
if src != last_addr:
    print(" " + describe(src, args.numeric, state["cache"]),
          end="")
    last_addr = src
print(f" {(arrival - start) * 1000:.3f} ms", end="", flush=True)
if icmp_type == ICMP_DEST_UNREACHABLE:
    done = True
    if icmp_code in UNREACH_FLAGS:
        print(f" {UNREACH_FLAGS[icmp_code]}", end="")
print(flush=True)
return done, lost

```

my_traceroute.**print_summary**(rows, nqueries)

Print the -S summary of unanswered probes per hop.

Parameters

- **rows** (*list*) – (ttl, unanswered) tuples.
- **nqueries** (*int*) – Probes sent per hop.

Source code:

```

def print_summary(rows, nqueries):
    """Print the ``-S`` summary of unanswered probes per hop.

    :param list rows: ``(ttl, unanswered)`` tuples.

```

(continues on next page)

(continued from previous page)

```

:param int nqueries: Probes sent per hop.
"""

print("\nSummary of unanswered probes:")
print("Hop Sent Unanswered")
for ttl, lost in rows:
    pct = 100 * lost // nqueries
    print(f"{ttl:3d} {nqueries:4d} {lost:4d} ({pct}%)")

```

my_traceroute.main()

Run the traceroute program.

Returns

Process exit status.

Return type

int

Source code:

```

def main():
    """Run the traceroute program.

    :return: Process exit status.
    :rtype: int
    """
    args = parse_args()
    try:
        dest_ip = resolve_host(args.host)
    except socket.gaierror:
        print(f"{args.host}: Name or service not known", file=sys.stderr)
        return 2
    try:
        recv_sock = socket.socket(socket.AF_INET, socket.SOCK_RAW,
                                   socket.IPPROTO_ICMP)
    except PermissionError:
        print("traceroute: raw sockets need root/administrator "
              "privileges (try sudo)", file=sys.stderr)
        return 1
    send_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM,
                               socket.IPPROTO_UDP)

    print(f"traceroute to {args.host} ({dest_ip}), {MAX_HOPS} hops max, "
          f"{PACKET_LEN} byte packets", flush=True)
    state = {"port": BASE_PORT, "cache": {}}
    rows = []
    try:
        for ttl in range(1, MAX_HOPS + 1):
            done, lost = trace_hop(ttl, dest_ip, args, send_sock,
                                   recv_sock, state)
            rows.append((ttl, lost))
            if done:

```

(continues on next page)

(continued from previous page)

```
        break
    except KeyboardInterrupt:
        print()
    finally:
        send_sock.close()
        recv_sock.close()

    if args.summary:
        print_summary(rows, args.nqueries)
    return 0
```

UTILITIES

Shared helpers for `my_ping.py` and `my_traceroute.py`.

This module contains the Internet checksum, ICMP packet construction and IP header parsing used by both programs.

`utils.checksum(data)`

Compute the 16-bit Internet checksum (RFC 1071).

Parameters

data (*bytes*) – Data to checksum.

Returns

The one's complement checksum.

Return type

`int`

Source code:

```
def checksum(data):
    """Compute the 16-bit Internet checksum (RFC 1071).

    :param bytes data: Data to checksum.
    :return: The one's complement checksum.
    :rtype: int
    """
    if len(data) % 2:
        data += b"\x00"
    total = 0
    for i in range(0, len(data), 2):
        total += (data[i] << 8) + data[i + 1]
    total = (total >> 16) + (total & 0xFFFF)
    total += total >> 16
    return ~total & 0xFFFF
```

`utils.build_echo_request(ident, seq, payload)`

Build an ICMP Echo Request packet.

Parameters

- **ident** (*int*) – ICMP identifier (usually derived from the PID).
- **seq** (*int*) – Sequence number.

- **payload** (*bytes*) – Data bytes appended after the ICMP header.

Returns

The complete ICMP packet with a valid checksum.

Return type

bytes

Source code:

```
def build_echo_request(ident, seq, payload):
    """Build an ICMP Echo Request packet.

    :param int ident: ICMP identifier (usually derived from the PID).
    :param int seq: Sequence number.
    :param bytes payload: Data bytes appended after the ICMP header.
    :return: The complete ICMP packet with a valid checksum.
    :rtype: bytes
    """
    header = struct.pack("!BBHH", ICMP_ECHO_REQUEST, 0, 0, ident, seq)
    csum = checksum(header + payload)
    header = struct.pack("!BBHH", ICMP_ECHO_REQUEST, 0, csum, ident, seq)
    return header + payload
```

`utils.parse_ip_header(data)`

Parse the fixed fields of an IPv4 header.

Parameters

data (*bytes*) – Raw packet beginning with an IPv4 header.

Returns

Tuple (header_length, ttl, source_ip).

Return type

tuple

Source code:

```
def parse_ip_header(data):
    """Parse the fixed fields of an IPv4 header.

    :param bytes data: Raw packet beginning with an IPv4 header.
    :return: Tuple ``(header_length, ttl, source_ip)``.
    :rtype: tuple
    """
    header_len = (data[0] & 0x0F) * 4
    ttl = data[8]
    src = socket.inet_ntoa(data[12:16])
    return header_len, ttl, src
```

`utils.resolve_host(host)`

Resolve a hostname or dotted address to an IPv4 address.

Parameters

host (*str*) – Hostname or IP address.

Returns

IPv4 address as a string.

Return type

str

Raises

socket.gaierror – If the name cannot be resolved.

Source code:

```
def resolve_host(host):  
    """Resolve a hostname or dotted address to an IPv4 address.  
  
    :param str host: Hostname or IP address.  
    :return: IPv4 address as a string.  
    :rtype: str  
    :raises socket.gaierror: If the name cannot be resolved.  
    """  
    return socket.gethostbyname(host)
```

PYTHON MODULE INDEX

m

`my_ping`, [1](#)

`my_traceroute`, [7](#)

U

`utils`, [14](#)

INDEX

B

`build_echo_request()` (*in module utils*), 14

C

`checksum()` (*in module utils*), 14

D

`describe()` (*in module my_traceroute*), 9

M

`main()` (*in module my_ping*), 4

`main()` (*in module my_traceroute*), 12

`make_payload()` (*in module my_ping*), 2

`module`

`my_ping`, 1

`my_traceroute`, 7

`utils`, 14

`my_ping`

`module`, 1

`my_traceroute`

`module`, 7

P

`parse_args()` (*in module my_ping*), 1

`parse_args()` (*in module my_traceroute*), 7

`parse_ip_header()` (*in module utils*), 15

`print_stats()` (*in module my_ping*), 3

`print_summary()` (*in module my_traceroute*), 11

`probe_port_from_icmp()` (*in module my_traceroute*), 7

R

`resolve_host()` (*in module utils*), 15

T

`trace_hop()` (*in module my_traceroute*), 10

U

`utils`

`module`, 14

W

`wait_for_icmp()` (*in module my_traceroute*), 8

`wait_for_reply()` (*in module my_ping*), 2